

Laboratorio 2: Programación en Radio Definida por Software (GNU Radio)

Andrés Felipe Flórez Quiroga - 2230399
Edgar Joel Lizarazo Perez - 2234046

https://github.com/Andr3s-F/2026_2_CommII_A1L

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Universidad Industrial de Santander

31 de agosto de 2026

Abstract—This report presents the implementation of custom signal processing blocks in GNU Radio through the Embedded Python Block interface. We built an accumulator and a differentiator with internal state, a five output block that computes running time averages, and a moving average filter designed by our team. The statistical block was validated against a deterministic periodic signal and then applied to a 200 Hz cosine corrupted by additive white Gaussian noise, where the moving average filter reduced that noise. Standard deviation measurements before and after filtering agreed with the analytical model within 3.0 percent for noise levels between 0.25 and 1.00, and within 2.1 percent when the averaging window varied from 4 to 64 samples.

Index Terms—GNU Radio, Embedded Python Block, promedios de tiempo, ruido gaussiano aditivo, promediado móvil, relación señal a ruido.

I Introducción

GNU Radio permite armar cadenas de procesamiento conectando bloques ya construidos, y también deja escribir bloques propios en Python [2]. Lo que más cuesta al empezar tiene que ver con cómo la herramienta ejecuta un diagrama: el planificador no entrega la señal completa de una sola vez, sino que llama repetidamente a la función `work()` con bloques de muestras cuyo tamaño depende de los buffers y de la carga del sistema. Un bloque sin memoria no sufre con eso. Uno que necesita recordar lo que pasó antes, como un acumulador, sí.

Los algoritmos base provienen del libro guía [1] y dos de ellos no corrían tal como estaban publicados. Este informe recoge esas correcciones, los cuatro bloques implementados y un promediador móvil de diseño propio con el que usamos la estadística para resolver un problema de ruido.

II Metodología

II-A Organización del repositorio

Trabajamos sobre una rama de integración llamada `Practica_1` y una rama individual por integrante, `P1_Edgar` y `P1_Andres`. Los archivos `.grc` se repartieron por experimento, ya que dos personas editando el mismo diagrama generan conflictos que la fusión automática no resuelve.

II-B Bloques implementados

El acumulador realiza la suma corrida $y[n] = y[n-1] + x[n]$ y el diferenciador la operación inversa, $y[n] = x[n] - x[n-1]$. En ambos la continuidad entre llamadas a `work()` depende de un atributo de la clase: el acumulador guarda el último valor acumulado y el diferenciador la última muestra de entrada. Sin ese atributo la salida se corta en cada frontera.

El tercer bloque calcula cinco promedios de tiempo: media, media cuadrática, valor RMS, potencia promedio y desviación estándar. Para señales reales $P = \overline{x^2} = V_{RMS}^2$, de modo que la cuarta y la segunda salida coinciden. Las conservamos separadas porque el enunciado pedía cinco.

El cuarto bloque es diseño nuestro y aplica un promedio móvil de N muestras,

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k]. \quad (1)$$

Su implementación usa `set_history(N)`, método de `gr.sync_block` que pide al planificador $N-1$ muestras adicionales del bloque anterior para que la ventana no se corte entre llamadas.

Listing 1: Bloque de promediado móvil implementado por el grupo.

```

1 def __init__(self, N=16):
2     gr.sync_block.__init__(self,
3         name='Promediador_Movil',
4         in_sig=[np.float32],
5         out_sig=[np.float32])
6     self.N = int(N)
7     self.set_history(self.N)
8 def work(self, input_items, output_items):
9     x = input_items[0]
10    y0 = output_items[0]
11    ventana = np.ones(self.N) / self.N
12    y0[:] = np.convolve(x, ventana, mode='valid')
13    return len(y0)

```

II-C Correcciones a los algoritmos base

Los algoritmos de la sección 1.2.0.1 del libro guía [1] requirieron los ajustes de la Tabla 1. Los dos primeros impedían la ejecución.

Tabla 1: Ajustes aplicados a los algoritmos del libro guía.

Bloque	Problema	Corrección
Acumulador	return len(y) con variable inexistente	Retornar la longitud del vector de salida
Diferenciador	Atributo asignado dentro de los paréntesis de __init__	Moverlo después de la llamada
Acumulador	cumsum sin estado, reinicia en cada llamada	Sumar el último acumulado guardado
Diferenciador	Resta del acumulado previo, que no es una derivada	np.insert más np.diff sobre la última muestra
Promedios	acumulado[N-1] asignado al acumulador cuadrático	Usar acumulado1[N-1]

II-D Montajes de prueba

Todos los diagramas corrieron a 32 kHz e incluyeron un bloque *Throttle*, necesario porque no hay hardware de radio que marque el ritmo de la simulación. Los montajes del acumulador y del diferenciador siguen la misma cadena: *Vector Source*, el bloque de Python, *Throttle* y *QT GUI Time Sink*. Los dos montajes con instrumentación estadística aparecen en la Figura 1.

La entrada del acumulador fue (5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5) repetida, elegida porque suma cero en un período y deja la salida acotada. La del diferenciador, (0, 1, 2, 3, 4, 5, 5, 5, 5, 4, 3, 2, 1, 0), una rampa que sube, se mantiene y baja. El bloque estadístico se validó con el vector (1, 2, -1) repetido, cuyos promedios se calculan a mano, y luego se aplicó a un coseno de 200 Hz y amplitud unitaria sumado con ruido gaussiano blanco de desviación ajustable.

El último montaje, en la Figura 2, agrega el promediador móvil. La señal ruidosa recorre dos caminos, uno directo y otro a través del filtro, y cada camino alimenta un bloque estadístico, de modo que la desviación antes y después se lee en la misma pantalla.

III RESULTADOS Y ANÁLISIS DE DATOS

III-A Acumulador y diferenciador

La suma corrida de la entrada produce los valores 5, 9, 12, 14, 15, 15, 14, 12, 9, 5 y 0, que es la forma triangular de la Figura 3a. El regreso a cero al final de cada período confirma que el estado se conserva entre llamadas, ya que de perderse la traza mostraría discontinuidades en las fronteras de los bloques de muestras.

En la Figura 3b la salida toma tres niveles: +1 mientras la entrada sube con pendiente unitaria, cero durante la meseta y -1 en el descenso. Un diferenciador responde a los cambios de la señal, no a su nivel, y la meseta lo deja en evidencia.

III-B Validación del bloque estadístico

Con la entrada determinística (1, 2, -1) los cinco promedios tienen valor conocido. La Tabla 2 compara ese cálculo con lo que reportó el bloque.

Tabla 2: Validación del bloque estadístico con la señal (1, 2, -1).

Parámetro	Teórico	Medido	Error (%)
Media	0.66667	0.666647	-0,003
Media cuadrática	2.00000	1.999929	-0,004
Valor RMS	1.41421	1.414157	-0,004
Potencia promedio	2.00000	1.999929	-0,004
Desviación estándar	1.24722	1.252151	+0,395

Cuatro de las cinco salidas coinciden hasta el quinto decimal. La quinta se desvía un 0.4 por ciento porque el algoritmo del libro acumula $(x[n] - \bar{x}[n])^2$ contra la media corrida, que cambia en cada muestra, cuando la definición usa una media fija. Al aplicar la identidad

$$\sigma^2 = \overline{x^2} - \bar{x}^2 \quad (2)$$

sobre las salidas medidas se obtiene $\sqrt{1,999929 - 0,666647^2} = 1,2472$, que reproduce el valor teórico. Por eso implementamos la ecuación (2) en el bloque de desviación usado más adelante.

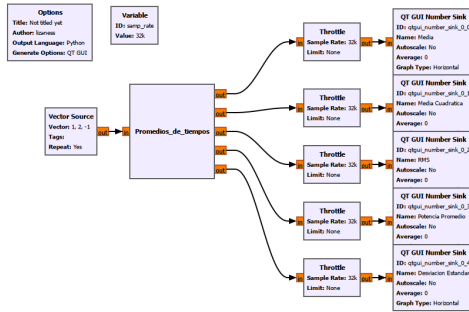
III-C Filtrado por promediado móvil

Para una señal de media nula la desviación total combina las dos contribuciones,

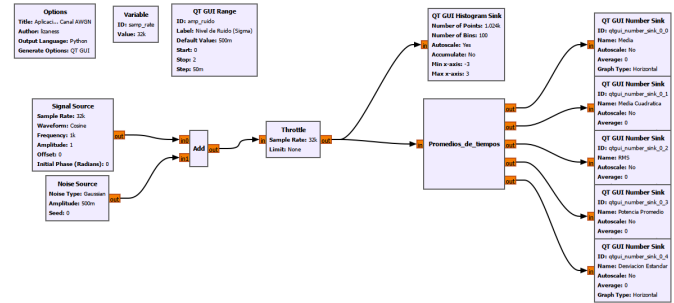
$$\sigma_{tot} = \sqrt{\frac{A^2}{2} + \sigma_r^2}, \quad (3)$$

con A la amplitud del coseno y σ_r la desviación del ruido. El promediador de la ecuación (1) se comporta como un filtro pasabajas de respuesta [3]

$$|H(f)| = \left| \frac{\sin(\pi N f / f_s)}{N \sin(\pi f / f_s)} \right|, \quad (4)$$



(a) Validación del bloque estadístico.



(b) Medición sobre señal con ruido gaussiano aditivo.

Figura 1: Diagramas de bloques de los montajes con instrumentación estadística.

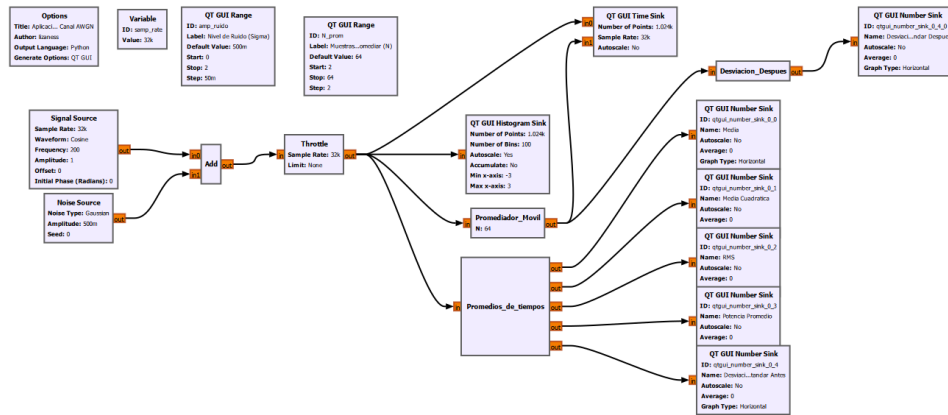
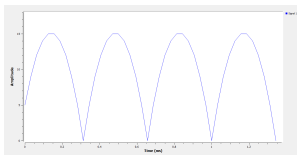
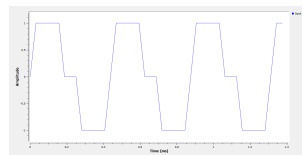


Figura 2: Montaje de filtrado por promediado móvil, con medición estadística en ambos caminos.



(a) Acumulador.



(b) Diferenciador.

Figura 3: Respuesta temporal de los bloques con memoria.

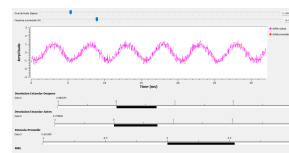
que atenúa también la señal deseada, de modo que la desviación esperada a la salida queda

$$\sigma_{fil} = \sqrt{\frac{A^2 |H(f_0)|^2}{2} + \frac{\sigma_r^2}{N}}. \quad (5)$$

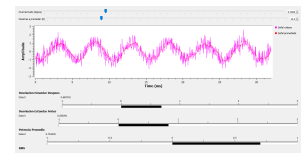
Con $f_0 = 200$ Hz y $f_s = 32$ kHz, la ecuación (4) da $|H| = 0,9990$ para $N = 4$, $0,9837$ para $N = 16$ y $0,7569$ para $N = 64$.

Realizamos dos series independientes. En la primera variamos el nivel de ruido con $N = 16$ fijo, y en la segunda dejamos $\sigma_r = 0,5$ y cambiamos el tamaño de

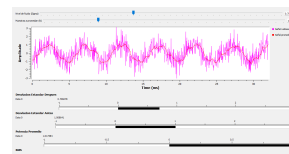
la ventana. Las Figuras 4 y 5 muestran las condiciones y la Tabla 3 compara las lecturas contra las ecuaciones (3) y (5).



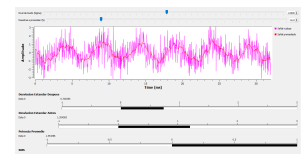
(a) $\sigma_r = 0,25$



(b) $\sigma_r = 0,50$



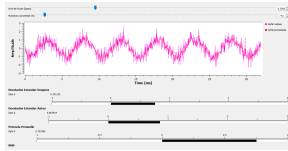
(c) $\sigma_r = 0,75$



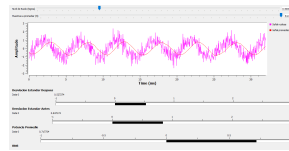
(d) $\sigma_r = 1,00$

Figura 4: Señal ruidosa y señal promediada para cuatro niveles de ruido, con $N = 16$.

Los errores se mantienen por debajo del 3.0 por ciento y casi todos apuntan en la misma dirección. Ese



(a) $N = 4$



(b) $N = 64$

Figura 5: Efecto del tamaño de la ventana, con $\sigma_r = 0,5$. El caso $N = 16$ aparece en la Figura 4b.

Tabla 3: Desviación estándar antes y después del filtrado en las dos series de medición.

Condición	Antes		Después		Err. (%)
	Teór.	Med.	Teór.	Med.	
Serie 1: ventana fija en $N = 16$					
$\sigma_r = 0,25$	0.7500	0.7359	0.6984	0.6852	-1,9
$\sigma_r = 0,50$	0.8660	0.8393	0.7067	0.6857	-3,0
$\sigma_r = 0,75$	1.0308	1.0086	0.7204	0.7061	-2,0
$\sigma_r = 1,00$	1.2247	1.2046	0.7391	0.7304	-1,2
Serie 2: ruido fijo en $\sigma_r = 0,5$					
$N = 4$	0.8660	0.8679	0.7494	0.7512	+0,2
$N = 16$	0.8660	0.8488	0.7067	0.6929	-1,9
$N = 64$	0.8660	0.8472	0.5388	0.5273	-2,1

sesgo no es aleatorio: el bloque incrementa el contador `Ntotales` con todo el bloque de muestras y luego divide todas las muestras entre el total, cuando la primera debería dividirse entre el conteo previo más uno. La subestimación se reduce conforme el conteo crece, y las lecturas se tomaron pocos segundos después de reiniciar cada corrida.

De ahí salió también una limitación práctica. Como los acumuladores nunca olvidan, mover el deslizador sin reiniciar contamina la medición con el historial del valor anterior, y en un primer intento las lecturas se apartaron hasta un 10 por ciento. Repetimos las cuatro condiciones reiniciando el diagrama en cada una y los errores cayeron al rango de la Tabla 3.

III-D Ganancia en relación señal a ruido

El promediado corresponde a la aplicación pedida: recuperar una señal enterrada en ruido. Con la potencia de señal $A^2|H|^2/2$ y la de ruido σ_r^2/N , la relación a la salida vale

$$\text{SNR}_{fil} = 10 \log_{10} \left(\frac{A^2|H(f_0)|^2 N}{2 \sigma_r^2} \right). \quad (6)$$

Evaluada sobre las condiciones de la Tabla 3, la ecuación (6) arroja una ganancia constante de 11.90 dB para los cuatro niveles de ruido con $N = 16$, frente a los 12.04 dB que predice $10 \log_{10}(N)$. Los 0.14 dB

de diferencia son la atenuación que el filtro aplica a la señal. Con $N = 4$ la ganancia es 6.01 dB contra 6.02 dB teóricos, y con $N = 64$ llega a 15.64 dB contra 18.06 dB, con una brecha de 2.42 dB que coincide con $10 \log_{10}(0,5729)$.

IV Conclusiones

- El acumulador y el diferenciador conservaron la continuidad de sus salidas entre bloques de muestras consecutivos, lo que confirma que el estado almacenado en atributos de la clase resuelve la fragmentación que impone el planificador.
- El bloque de promedios de tiempo reprodujo la media, la media cuadrática, el valor RMS y la potencia promedio de la señal (1, 2, -1) con errores inferiores al 0.004 por ciento. La desviación estándar se apartó un 0.395 por ciento por acumular contra una media corrida en lugar de una media fija.
- El promediado de N muestras redujo la desviación del ruido en un factor cercano a $\sqrt{N}$, con errores por debajo del 3.0 por ciento respecto al modelo analítico.
- Los acumuladores sin factor de olvido solo entregan valores válidos si el sistema permanece estacionario desde el arranque. Para medir mientras un parámetro cambia convendría una versión con ventana deslizante o con olvido exponencial.
- La ganancia en relación señal a ruido llegó a 11.90 dB con $N = 16$ contra 12.04 dB teóricos, y la brecha creció hasta 2.42 dB con 64 muestras. La diferencia es la atenuación sobre la señal deseada, de modo que la ventana no puede elegirse solo por cuánto ruido elimina.

Referencias

- [1] Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones, *Libro de Comunicaciones Plus Volumen I*. Bucaramanga, Colombia: Universidad Industrial de Santander, 2020, material de apoyo académico.
- [2] GNU Radio Project, "Creating Your First Block," GNU Radio Wiki. [En línea]. Disponible: https://wiki.gnuradio.org/index.php?title=Creating_Your_First_Block
- [3] A. V. Oppenheim y R. W. Schaffer, *Discrete-Time Signal Processing*, 3a ed. Upper Saddle River, NJ, EE. UU.: Pearson, 2010.
- [4] O. J. Tijero Rojas, *Programación en Radio Definida por Software (GNURadio)*, guía de laboratorio, Comunicaciones II (27145). Universidad Industrial de Santander, 2026.